

GROUP B - Backend & AI/ML Team Guide

M3: FastAPI + Database | M4: RAG + MCQ + LightGBM

What This Guide Covers

- Complete project overview and your team's exact role
- All topics to learn with free online resource links
- Step-by-step environment & tool installation guide
- Google Antigravity (AGY) usage with example prompts
- Week-by-week 16-week build plan with deliverables
- Integration contracts with other groups
- Testing, common mistakes, and daily workflow

Primary Development Tool: Google Antigravity (AGY) IDE

AI Learning Platform - GROUP B (Backend + AI/ML) Team Guide

Project: AI-Enabled Learning & Competency Gap Platform for India's Official Statistical System (SIH 2026)

Members:

- M3: Backend Developer (FastAPI, PostgreSQL, Redis, Auth, Core API)
- M4: AI/ML Developer (RAG, Gemini API, Embeddings, Recommendations)

1. PROJECT OVERVIEW

The AI-Enabled Learning & Competency Gap Platform is designed to modernize the official statistical system by identifying competency gaps in personnel and recommending targeted learning modules. The platform leverages AI for intelligent recommendations and dynamically generates assessments to evaluate proficiency.

Group B's Role:

You are the brain of the operation. Group B is responsible for the entire backend architecture, database modeling, REST API, authentication, caching, AI services integration, document ingestion, and recommendation algorithms.

How AI/ML Fits:

AI/ML is central to this platform:

- RAG Pipeline: For querying and indexing massive troves of statistical literature and documents.
- Gemini MCQ Generation: Automating the creation of context-aware multiple-choice questions for assessments.
- Recommendation Engine: Suggesting learning pathways based on competency gaps, user profile, and historical data.

2. MEMBER RESPONSIBILITIES

M3 (Backend Developer)

Your focus is building a robust, high-performance API server and data layer.

- FastAPI App: Develop the core application framework with dependency injection and middleware.
- PostgreSQL Schema: Design and implement all 12 core entities.
- JWT Auth: Secure all endpoints with role-based access control.
- Gap Engine Formula: Implement the core business logic for competency gap calculation.
- iGOT Mock Adapter: Build the interface to interact with the iGOT platform.
- Redis Caching: Implement caching for heavy read operations.
- Audit Logging: Ensure all actions are auditable.
- pytest Tests: Write comprehensive unit and integration tests.

M4 (AI/ML Developer)

Your focus is intelligent features and data pipelines.

- RAG Pipeline: Implement document chunking, embeddings, and vector search.
- Document Ingestion: Parse PDFs, DOCX, PPTX, and perform OCR (pytesseract).
- Gemini MCQ Generation: Prompt engineering for strict JSON schema MCQs.
- MCQ Validator: Validate MCQs for answerability and distractors.
- Recommendation Engine:
 - Phase 1: Weighted formula.
 - Phase 2: LightGBM.
-

Multilingual Embeddings: Use multilingual-e5-large for diverse language support.

- MLflow Tracking: Track experiments and model versions.

3. TOPICS TO LEARN (with free URLs)

Fundamentals & Backend

- Python Fundamentals: python.org/tutorial - Ensure you understand async programming and type hints.
- FastAPI: fastapi.tiangolo.com - Learn path operations, dependencies, and validation.
- PostgreSQL + pgvector: postgresql.org/docs | github.com/pgvector/pgvector
- SQLAlchemy 2.0 ORM: docs.sqlalchemy.org - Focus on 2.0 style queries.
- Alembic migrations: alembic.sqlalchemy.org
- Redis basics: redis.io/docs
- JWT authentication: jwt.io
- pytest: docs.pytest.org

AI & Data Science

- Google Gemini API: ai.google.dev - Understand the SDK and prompting strategies.
- RAG Architecture: Read Pinecone's RAG guide .
- Embeddings and Vector Search: HuggingFace docs for sentence-transformers.
- LightGBM: lightgbm.readthedocs.io
- MLflow: mlflow.org/docs
- PDF/DOCX parsing: Check pdfplumber, python-docx, pytesseract docs.

4. ENVIRONMENT SETUP (step-by-step)

Step 1: Python 3.12 Install

Download and install Python 3.12 from python.org. Ensure it's added to PATH.

Step 2: Virtual Environment

```
python -m venv venv
venv\Scripts\activate
```

Step 3: Install Dependencies

```
pip install fastapi "uvicorn[standard]" sqlalchemy alembic psycpg2-binary asyncpg
pip install passlib[bcrypt] python-jose[cryptography] python-multipart
pip install redis pytest pytest-asyncio httpx
pip install google-generativeai sentence-transformers pdfplumber python-docx python-pptx pytesseract
pip install lightgbm scikit-learn mlflow numpy pandas pgvector
```

Step 4: PostgreSQL + pgvector Setup

Install PostgreSQL 16. Use Docker to easily run with pgvector:

```
docker run --name postgres-pgvector -e POSTGRES_PASSWORD=secret -e POSTGRES_USER=sih -e POSTGRES_DB=sih_db -p 5432:
```

Step 5: Redis Setup

```
docker run --name redis-server -p 6379:6379 -d redis
```

Step 6: .env Configuration

Create a .env file in the root directory:

```
DATABASE_URL=postgresql+asyncpg://sih:secret@localhost:5432/sih_db
REDIS_URL=redis://localhost:6379/0
SECRET_KEY=your_super_secret_key
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
GEMINI_API_KEY=your_gemini_api_key
```

5. ANTIGRAVITY USAGE GUIDE

Google Antigravity (AGY) is your primary development IDE. It supports agentic capabilities.

Example Prompts:

- FastAPI Routers: "Generate a FastAPI router for user management with endpoints for CRUD operations. Use standard dependency injection for the DB session."
- Pydantic Models: "Create Pydantic v2 models for the Competency entity, including ConfigDict for from_attributes."
- Gap Engine: "Implement a python function to calculate the competency gap score based on the formula: $0.40\text{gap_match} + 0.20\text{level_fit} + 0.15\text{duration_fit} + 0.10\text{evidence_fit} + 0.10\text{language_fit} + 0.05\text{freshness}$."
- RAG Pipeline: "Write a Python script using sentence-transformers (multilingual-e5-large) to embed a list of texts and store them in PostgreSQL using pgvector and SQLAlchemy."
- Gemini MCQ: "Create a prompt template for Gemini API to generate 5 MCQs from the given context. The output must strictly follow this JSON schema: `[{'question': str, 'options': list[str], 'answer': str, 'explanation': str}]`."

Limitations:

- Always review code for business logic accuracy.
- AGY might use older Pydantic v1 syntax; explicitly specify v2.

6. COMPLETE FOLDER STRUCTURE

```

C:\Users\user\SIH\
??? alembic.ini
??? .env
??? requirements.txt
??? services/
?   ??? api/
?   ?   ??? main.py
?   ?   ??? core/
?   ?   ?   ??? config.py
?   ?   ?   ??? security.py
?   ?   ?   ??? database.py
?   ?   ??? models/           # SQLAlchemy models
?   ?   ??? schemas/         # Pydantic models
?   ?   ??? api/
?   ?   ?   ??? v1/
?   ?   ?   ?   ??? endpoints/
?   ?   ?   ?   ??? api.py
?   ?   ?   ??? dependencies.py
?   ?   ??? crud/             # Database operations
?   ?   ??? engine/           # Gap engine logic
?   ?   ??? adapters/         # iGOT adapter
?   ??? ai/
?   ?   ??? ingestion/
?   ?   ?   ??? parsers.py
?   ?   ?   ??? chunker.py
?   ?   ??? embeddings/
?   ?   ?   ??? generate.py
?   ?   ??? rag/
?   ?   ?   ??? retriever.py

```

7. DATABASE SCHEMA

Users Table

```

CREATE TABLE users (
    id UUID PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    hashed_password VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL,
    department VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_users_email ON users(email);

```

Competencies Table

```

CREATE TABLE competencies (
    id UUID PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    category VARCHAR(100)
);

```

UserCompetencies Table

```
CREATE TABLE user_competencies (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  competency_id UUID REFERENCES competencies(id),
  current_level INT CHECK (current_level >= 1 AND current_level <= 5),
  target_level INT CHECK (target_level >= 1 AND target_level <= 5),
  last_assessed TIMESTAMP
);
```

LearningModules Table

```
CREATE TABLE learning_modules (
  id UUID PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  duration_hours FLOAT,
  language VARCHAR(50),
  content_vector vector(1024) -- pgvector column
);
```

(Additional tables: Assessments, MCQQuestions, ModuleCompetencies, AuditLogs, iGOTMappings, UserRecommendations, Feedbacks, SystemSettings)

8. GAP ENGINE

The Gap Engine calculates relevance scores for learning modules.

Formula:

$$\text{score} = 0.40\text{gap_match} + 0.20\text{level_fit} + 0.15\text{duration_fit} + 0.10\text{evidence_fit} + 0.10\text{language_fit} + 0.05\text{freshness}$$

Python Implementation:

```
def calculate_recommendation_score(
    gap_match: float,
    level_fit: float,
    duration_fit: float,
    evidence_fit: float,
    language_fit: float,
    freshness: float
) -> float:
    return (0.40 * gap_match) + \
           (0.20 * level_fit) + \
           (0.15 * duration_fit) + \
           (0.10 * evidence_fit) + \
           (0.10 * language_fit) + \
           (0.05 * freshness)
```

Test Cases:

```
def test_gap_engine_perfect_match():
    score = calculate_recommendation_score(1.0, 1.0, 1.0, 1.0, 1.0, 1.0)
    assert abs(score - 1.0) < 1e-6

def test_gap_engine_partial_match():
    score = calculate_recommendation_score(0.5, 0.8, 0.2, 0.9, 1.0, 0.5)
    expected = (0.40*0.5) + (0.20*0.8) + (0.15*0.2) + (0.10*0.9) + (0.10*1.0) + (0.05*0.5)
    assert abs(score - expected) < 1e-6
```

9. FASTAPI ROUTERS

1. Auth Router: /api/v1/auth - Login, Token generation.
2. Users Router: /api/v1/users - CRUD for users.
3. Competencies Router: /api/v1/competencies - Manage dictionary.
4. UserCompetencies Router: /api/v1/users/{user_id}/competencies
5. Modules Router: /api/v1/modules - Manage learning materials.
6. Assessments Router: /api/v1/assessments - Trigger tests.
7. Recommendations Router: /api/v1/recommendations - Get gap engine results.

(Additional deep dive routers below)

10. RAG PIPELINE

Chunking Strategy

- Size: 300-800 tokens.
- Overlap: 50-100 tokens to maintain context.

Embeddings

Using multilingual-e5-large.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('intfloat/multilingual-e5-large')

def get_embedding(text: str) -> list[float]:
    # e5 models require 'query: ' or 'passage: ' prefix
    return model.encode(f"passage: {text}").tolist()
```

Hybrid Retrieval

Combine dense vector search (pgvector ORDER BY content_vector <-> query_vector) with sparse search (PostgreSQL Full Text Search).

11. MCQ GENERATION

Gemini API Integration

```

import google.generativeai as genai
import json

genai.configure(api_key="YOUR_API_KEY")
model = genai.GenerativeModel('gemini-2.0-flash')

def generate_mcqs(context: str, lang: str = "English"):
    prompt = f"""
    Generate 3 MCQs based on the context below in {lang}.
    Output MUST be a raw JSON array, with no markdown formatting.
    Schema:
    [
        {{
            "question": "string",
            "options": ["str1", "str2", "str3", "str4"],
            "answer": "str1",
            "explanation": "string"
        }}
    ]
    Context: {context}
    """

    response = model.generate_content(prompt)
    return json.loads(response.text)

```

Validator

Check if the answer is in options, and if options are distinct.

12. RECOMMENDATION ENGINE

Phase 1: Weighted Formula

Implemented in the Gap Engine section.

Phase 2: LightGBM

Train a learning-to-rank model using user interaction history.

```

import lightgbm as lgb
import mlflow

def train_recommender(X_train, y_train):
    mlflow.set_experiment("Recommendation_Engine")
    with mlflow.start_run():
        train_data = lgb.Dataset(X_train, label=y_train)
        params = {'objective': 'regression', 'metric': 'rmse'}

        model = lgb.train(params, train_data, num_boost_round=100)
        mlflow.lightgbm.log_model(model, "lgb_model")
        return model

```

13. iGOT MOCK ADAPTER

Create an abstract interface for the external iGOT system.


```

from abc import ABC, abstractmethod

class IGotAdapter(ABC):
    @abstractmethod
    async def fetch_user_courses(self, user_id: str):
        pass

class MockIGotAdapter(IGotAdapter):
    async def fetch_user_courses(self, user_id: str):
        return [{"course_id": "c1", "status": "completed"}]

```

14. WEEK-BY-WEEK TASKS (16 Weeks)

Week 1-2: Project setup, DB schema design, FastAPI skeleton.

Week 3-4: Auth implementation, core CRUD APIs.

Week 5-6: Document ingestion pipeline, OCR setup.

Week 7-8: Embeddings, RAG pipeline integration.

Week 9-10: Gap Engine (Phase 1) and Recommendation API.

Week 11-12: Gemini MCQ generation and Assessment API.

Week 13-14: MLflow setup, LightGBM (Phase 2) training.

Week 15: iGOT adapter integration, comprehensive testing.

Week 16: Final optimizations, documentation, and deployment.

15. INTEGRATION WITH OTHER GROUPS

Provide Group A (Frontend) with Swagger UI (/docs). Mock API responses using FastAPI's `response_model` and dummy data so they can build the UI concurrently.

16. TESTING GUIDE

Run tests using pytest:

```
pytest tests/ -v --cov=services
```

Create a Golden Dataset for AI evaluations: A fixed set of documents and expected MCQs/Recommendations to measure prompt regressions.

17. COMMON MISTAKES TO AVOID

- Blocking the Event Loop: Do not run heavy ML inference or PDF parsing in FastAPI's main async thread. Use `asyncio.to_thread` or a background task queue (like Celery/Redis RQ).
- Missing Indexes: Always index foreign keys in PostgreSQL.
- Ignoring API Rate Limits: Handle Gemini API rate limits with exponential backoff.
- Poor Prompting: Not enforcing JSON schemas strictly in Gemini prompts.

18. DAILY WORKFLOW

1. Pull latest from main.
 2. Create a feature branch (git checkout -b feat/gap-engine).
 3. Write tests first (TDD).
 4. Implement feature.
 5. Ask AntigraVity for code review.
 6. Run pytest.
 7. Push and create PR.
-

19. RESOURCES & REFERENCE LINKS

- FastAPI Dependency Injection
 - pgvector Quickstart
 - LightGBM Python API
 - MLflow Quickstart
-

EXTENSIVE DETAILS & DEEP DIVES

A. Deep Dive: SQLAlchemy 2.0 and pgvector Models

SQLAlchemy 2.0 uses type-annotated Base models. Let's see how you declare the LearningModules table.

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy.types import String, Float, Text
from sqlalchemy import UUID
from pgvector.sqlalchemy import Vector
import uuid

class Base(DeclarativeBase):
    pass

class LearningModule(Base):
    __tablename__ = 'learning_modules'

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    title: Mapped[str] = mapped_column(String(255))
    description: Mapped[str] = mapped_column(Text)
    duration_hours: Mapped[float] = mapped_column(Float)
    language: Mapped[str] = mapped_column(String(50))
    content_vector: Mapped[list[float]] = mapped_column(Vector(1024))
```

B. Deep Dive: Advanced Recommendation Engine Feature Engineering

Feature engineering is critical for the Phase 2 LightGBM model.

Features to extract:

1. user_historical_completion_rate: Ratio of completed vs started modules.
2. module_avg_rating: Average user rating for the module.
3. skill_overlap_ratio: Percentage of skills in the module that match the user's competency gap.
4. time_since_last_learning: Days since the user last completed a module.

C. Deep Dive: Exception Handling in FastAPI

Create custom exception handlers for a unified response structure.

```

from fastapi import Request, JSONResponse
from fastapi.responses import JSONResponse

class BusinessLogicException(Exception):
    def __init__(self, name: str, detail: str):
        self.name = name
        self.detail = detail

async def business_logic_exception_handler(request: Request, exc: BusinessLogicException):
    return JSONResponse(
        status_code=400,
        content={"message": f"Business Rule Violation: {exc.name}", "detail": exc.detail},
    )

```

D. Deep Dive: Pytest Setup with Database Teardown

Testing against a real database requires cleanup. Use nested transactions or truncate tables.

```

import pytest
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker

@pytest.fixture
async def db_session():
    engine = create_async_engine("postgresql+asyncpg://sih:secret@localhost:5432/test_db")
    SessionLocal = async_sessionmaker(autocommit=False, autoflush=False, bind=engine)
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

    async with SessionLocal() as session:
        yield session

    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)

```

E. Deep Dive: Redis Caching Middleware

Cache frequent GET requests like competency dictionaries.

```

import redis.asyncio as redis
from fastapi import Request, Response

redis_client = redis.from_url("redis://localhost:6379")

async def cache_middleware(request: Request, call_next):
    if request.method == "GET":
        cache_key = request.url.path
        cached_data = await redis_client.get(cache_key)
        if cached_data:
            return Response(content=cached_data, media_type="application/json")

    response = await call_next(request)
    # logic to store response in cache goes here
    return response

```

F. Deep Dive: JWT Authentication Flow

1. User sends /login with username and password.
2. Backend verifies hash with passlib.
3. Backend signs a JWT using python-jose with a 30-minute expiry.
4. User attaches Authorization: Bearer <token> to requests.

5. FastAPI Depends(oauth2_scheme) extracts and decodes it.

```
from jose import jwt
from datetime import datetime, timedelta

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=30)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, "SUPER_SECRET", algorithm="HS256")
    return encoded_jwt
```

G. Deep Dive: Batch Ingestion Strategy for Large Corpora

When dealing with 10,000+ PDFs:

- Use multiprocessing to parse PDFs in parallel.
- Push text chunks to a Redis Queue.
- Background workers pick up chunks, call the Embedding API, and batch insert (500 chunks at a time) into PostgreSQL.

H. Deep Dive: Frontend Integration Contracts (Group A)

Example Contract for Gap Results (GET /api/v1/users/{id}/gap-report):

```
{
  "user_id": "uuid",
  "overall_gap_score": 0.75,
  "top_missing_competencies": [
    {
      "name": "Data Validation",
      "current_level": 2,
      "target_level": 4,
      "recommended_modules": [
        {"module_id": "uuid", "title": "Advanced Data Validation in R"}
      ]
    }
  ]
}
```

I. Deep Dive: Monitoring and Logging

Configure standard Python logging to output JSON. Use Elasticsearch + Kibana (ELK) or Grafana Loki to ingest these logs for debugging in production.

```
import logging
import json

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_record = {
            "time": self.formatTime(record, self.datefmt),
            "level": record.levelname,
            "message": record.getMessage(),
            "module": record.module
        }
        return json.dumps(log_record)
```

End of Document